

MODULE-5

CORE PHP

Q1. Discuss the structure of a PHP script and how to embed PHP in HTML.

1. Structure of a PHP Script

A basic PHP script has the following structure:

```
<?php
// PHP code starts here

// Variable declaration
$message = "Hello, World!";

// Function definition
function greet($name) {
    return "Welcome, " . $name . "!";
}

// Output
echo $message;
echo "<br>";
echo greet("Dipen");

// PHP code ends here
?>
```

Key Points:

- **<?php ... ?>:** Every PHP script must start with <?php and usually ends with ?>.
 - **Statements:** Each PHP statement ends with a semicolon ;.
 - **Comments:**
 - Single-line: // This is a comment
 - Multi-line: /* This is a multi-line comment */
 - **Case sensitivity:** PHP keywords (if, echo, while) are **not case-sensitive**, but variables are **case-sensitive**.
-

2. Embedding PHP in HTML

PHP is most powerful when used inside HTML to make web pages dynamic.

Example 1: Mixing HTML and PHP

```
<!DOCTYPE html>
<html>
<head>
    <title>PHP in HTML</title>
</head>
<body>
    <h1>Welcome to My Website</h1>
    <p>
        <?php
            $name = "Dipen";
            echo "Hello, " . $name . "! This content is generated by PHP.";
        ?>
    </p>
</body>
</html>
```

Here, PHP code runs on the server, and only the **output** (HTML) is sent to the browser.

Example 2: Short PHP Tags (if enabled in `php.ini`)

```
<p>Today's date is: <?= date("Y-m-d"); ?></p>
```

- `<?= ... ?>` is shorthand for `<?php echo ... ?>`.
-

Example 3: PHP Controlling HTML Output

```
<ul>
<?php
$fruits = ["Apple", "Banana", "Mango"];
foreach($fruits as $fruit) {
    echo "<li>$fruit</li>";
}
?>
</ul>
```

Q2. What are the rules for naming variables in PHP?

Rules for PHP Variable Names

1. **Start with \$ sign**
 - All variables in PHP must begin with a **dollar sign (\$)**.

2. `$name = "Dipen";`
 3. `$age = 25;`
 4. **Must start with a letter or underscore (_)**
 - o After \$, the first character must be a **letter (A–Z, a–z)** or an **underscore _**.
 - o **✗ Invalid:** `$9number = 100;`
 - o **✓ Valid:** `$number9 = 100;;`, `$_value = 50;`
 5. **Can contain letters, numbers, and underscores**
 - o You can use letters (a–z, A–Z), digits (0–9), and underscores (_) after the first character.
 6. `$user_name = "admin";`
 7. `$marks2025 = 95;`
 8. **Case-sensitive**
 - o `$name`, `$Name`, and `$NAME` are considered **different variables**.
 9. `$name = "Dipen";`
 10. `$Name = "Patel";`
 11. `echo $name; // Dipen`
 12. `echo $Name; // Patel`
 13. **No special characters or spaces**
 - o You **cannot** use characters like -, @, #, !, or spaces in variable names.
 - o **✗** `$user-name`, `$user@id`, `$first name` (invalid)
 14. **Follow naming conventions (good practice, not strict rule)**
 - o Use **camelCase** or **snake_case** for readability.
 - o Examples: `$firstName`, `$last_name`, `$studentAge`
-

Examples of Valid Variable Names

```
$age = 21;
$_total = 500;
$username = "Dipen";
$student_id = 12345;
```

Examples of Invalid Variable Names

```
$9count = 10;           // starts with a number
$first-name = "A";      // contains a hyphen
$my value = 5;          // contains a space
```

Q3. Explain the concept of variables in PHP and their scope

1. Concept of Variables in PHP

A **variable** in PHP is a container used to store data that can change during the execution of a script.

Key Features:

- Variables in PHP **start with a \$ sign**.
- They are **loosely typed** → you don't need to declare a type (PHP automatically assigns one based on the value).
- The value stored in a variable can be changed anytime.

Example:

```
<?php
$name = "Dipen";      // String
$age = 25;             // Integer
$height = 5.9;         // Float
$isStudent = true;    // Boolean

echo "My name is $name, I am $age years old.";
?>
```

2. Scope of Variables in PHP

The **scope** of a variable defines **where it can be accessed** in a script.

PHP has **4 main types of variable scope**:

a) Local Scope

- Variables declared **inside a function** are local to that function.
- They cannot be accessed outside.

```
<?php
function testLocal() {
    $x = 10; // local variable
    echo "Inside function: $x<br>";
}
testLocal();
// echo $x; // ✗ Error: Undefined variable
?>
```

b) Global Scope

- Variables declared **outside functions** are global.
- They cannot be accessed directly inside a function.
- Use the **global keyword** to access them.

```
<?php
$x = 50; // global variable

function testGlobal() {
    global $x; // import global variable
    echo "Inside function: $x<br>";
}

testGlobal();
echo "Outside function: $x<br>";
?>
```

c) Static Scope

- Normally, a local variable is destroyed after a function ends.
- If you declare it as **static**, it preserves its value between function calls.

```
<?php
function counter() {
    static $count = 0; // static variable
    $count++;
    echo "Count: $count<br>";
}

counter(); // Count: 1
counter(); // Count: 2
counter(); // Count: 3
?>
```

d) Super global Scope

- PHP provides some **built-in global arrays** that are accessible everywhere (inside and outside functions).
- These are called **super globals**.
- Examples:
 - `$_GET` → data from URL
 - `$_POST` → data from form submissions
 - `$_SERVER` → server info
 - `$_SESSION`, `$_COOKIE`, `$_FILES`, `$_ENV`, `$_REQUEST`

```
<?php
```

```
echo $_SERVER['PHP_SELF']; // Outputs current script name
?>
```

Q4. What are super global variables in PHP? List at least five super global arrays and their use

Super Global Variables in PHP

- Superglobals are **built-in variables** in PHP that are **always accessible**, regardless of scope (inside or outside a function).
 - You don't need to use `global` to access them.
 - They are mainly **associative arrays** that hold important information about user input, server environment, sessions, cookies, etc.
-

Common Superglobal Arrays in PHP

1. `$_GET`

- Holds data sent via **URL parameters (query string)**.
- Example URL: `index.php?name=Dipen&age=25`

```
<?php
echo "Name: " . $_GET['name']; // Dipen
echo "Age: " . $_GET['age']; // 25
?>
```

✓ Use: Collecting form data using **GET method** or reading URL query strings.

2. `$_POST`

- Holds data sent via **HTML form with POST method**.

```
<form method="post">
  Name: <input type="text" name="username">
  <input type="submit">
</form>
```

```
<?php
echo "Username: " . $_POST['username'];
?>
```

✓ Use: Collecting secure form data (not visible in URL).

3. \$_REQUEST

- Contains data from **both GET and POST**, and also cookies depending on server settings.

```
<?php
echo $_REQUEST['username']; // works for both GET and POST
?>
```

✓ Use: Quick way to fetch form data, but less secure than using \$_POST or \$_GET directly.

4. \$_SERVER

- Contains information about the server and execution environment.

```
<?php
echo $_SERVER['PHP_SELF'];      // Current script name
echo $_SERVER['SERVER_NAME'];   // Server host name
echo $_SERVER['REQUEST_METHOD']; // GET or POST
?>
```

✓ Use: Getting script path, server details, and request method.

5. \$_SESSION

- Stores data across **multiple pages** for a user (session variables).

```
<?php
session_start();
$_SESSION['user'] = "Dipen";
echo $_SESSION['user']; // Dipen
?>
```

Q5. Explain how conditional statements work in PHP

Conditional Statements in PHP

Conditional statements are used to **make decisions** in a program — they execute different blocks of code depending on whether a condition is **true** or **false**.

1. if Statement

Executes code **only if** the condition is true.

```
<?php
$age = 20;

if ($age >= 18) {
    echo "You are eligible to vote.";
}
?>
```

✓ Output: You are eligible to vote. (only if \$age >= 18)

2. if...else Statement

Provides an **alternative block** if the condition is false.

```
<?php
$age = 16;

if ($age >= 18) {
    echo "You can vote.";
} else {
    echo "You are not eligible to vote.";
}
?>
```

✓ Output: You are not eligible to vote.

3. if...elseif...else Statement

Checks **multiple conditions** in sequence.

```
<?php
$marks = 75;
```

```
if ($marks >= 90) {
    echo "Grade A";
} elseif ($marks >= 75) {
    echo "Grade B";
} elseif ($marks >= 50) {
    echo "Grade C";
} else {
    echo "Fail";
}
?>
```

✓ Output: Grade B

4. Nested if Statements

You can place an `if` inside another `if`.

```
<?php
$age = 25;
$citizen = true;

if ($age >= 18) {
    if ($citizen) {
        echo "Eligible to vote.";
    } else {
        echo "Not a citizen, cannot vote.";
    }
}
?>
```

5. switch Statement

Used when you need to compare a variable against **many fixed values**.

```
<?php
$day = "Tue";

switch ($day) {
    case "Mon":
        echo "Today is Monday";
        break;
    case "Tue":
        echo "Today is Tuesday";
        break;
    case "Wed":
        echo "Today is Wednesday";
        break;
    default:
```

```
        echo "Invalid day";
    }
?>
```

Output: Today is Tuesday

Q6. Discuss the difference between for loop, for each loop, and do-while loop in PHP.

1. for Loop

- Used when you **know in advance** how many times to repeat the loop.
- Syntax:

```
for (initialization; condition; increment/decrement) {
    // Code to execute
}
```

✓ Example:

```
for ($i = 1; $i <= 5; $i++) {
    echo "Number: $i<br>";
}
```

☞ Output: 1 to 5

2. for each Loop

- Special loop used **only for arrays and objects**.
- Iterates through each element without needing a counter.
- Syntax:

```
foreach ($array as $value) {
    // Code
}
```

With key and value:

```
foreach ($array as $key => $value) {
    // Code
}
```

✓ Example:

```
$fruits = ["Apple", "Banana", "Mango"];

foreach ($fruits as $fruit) {
    echo "$fruit<br>";
}
```

☞ Output: Apple, Banana, Mango

3. do-while Loop

- Similar to `while`, but ensures the block runs **at least once**, even if the condition is false.
- Syntax:

```
do {
    // Code to execute
} while (condition);
```

✓ Example:

```
$i = 1;
do {
    echo "Value: $i<br>";
    $i++;
} while ($i <= 3);
```

☞ Output: Value: 1, Value: 2, Value: 3

Q7. Define arrays in PHP. What are the different types of arrays?

1. Definition of Arrays in PHP

An **array** in PHP is a special variable that can store **multiple values** under a single name.

- Instead of creating many variables (`$fruit1`, `$fruit2`, ...), you use one array.
- Each value in an array is called an **element**.
- Each element is identified by a **key** (index).

☞ Arrays in PHP are actually **ordered maps**, meaning they associate *keys* with *values*.

2. Types of Arrays in PHP

There are **three main types**:

a) Indexed Array

- Stores values with **numeric index** (starts from 0 by default).
- Keys are numbers.

✓ Example:

```
<?php
$fruits = ["Apple", "Banana", "Mango"]; // Short form
// or
$fruits = array("Apple", "Banana", "Mango"); // Old form

echo $fruits[0]; // Apple
echo $fruits[1]; // Banana
?>
```

☞ Used when you just need a list of items.

b) Associative Array

- Uses **named keys** (strings) instead of numbers.

✓ Example:

```
<?php
$marks = [
    "Dipen" => 90,
    "Ravi" => 85,
    "Amit" => 78
];

echo "Dipen scored " . $marks["Dipen"]; // 90
?>
```

☞ Used when you want to store **key-value pairs** (like a dictionary).

c) Multidimensional Array

- An array containing one or more arrays inside it.
- Can represent **tables, matrices, or structured data**.

✓ Example:

```
<?php
$students = [
    ["Name" => "Dipen", "Age" => 21, "Marks" => 90],
    ["Name" => "Ravi", "Age" => 22, "Marks" => 85],
    ["Name" => "Amit", "Age" => 20, "Marks" => 78]
];

echo $students[0]["Name"]; // Dipen
echo $students[1]["Marks"]; // 85
?>
```

☞ Used when dealing with **complex data structures** (like JSON, tables, etc.).

Q8. What is the header function in PHP and how is it used?

- The **header()** function is used to **send raw HTTP headers** to the browser **before any output is sent**.
- It lets you control things like:
 - Page redirection
 - Content type (e.g., HTML, JSON, PDF)
 - File downloads
 - Cache control

☞ Since headers must be sent **before any HTML output**, you cannot use `header()` after printing anything (including whitespace).

2. Syntax

```
header(string $header, bool $replace = true, int $http_response_code = 0);
```

- **\$header** → The header string to send.
 - **\$replace** (optional) → Whether to replace a previous similar header (default = `true`).
 - **\$http_response_code** (optional) → Set HTTP response status code (like 200, 404, 301).
-

3. Common Uses of `header()`

a) Redirecting to Another Page

```
<?php
header("Location: welcome.php");
exit(); // Always use exit after redirect
?>
```

b) Setting Content-Type

```
<?php
header("Content-Type: application/json");
$data = ["name" => "Dipen", "age" => 25];
echo json_encode($data);
?>
```

☞ Browser interprets the output as **JSON**, not HTML.

c) Forcing File Download

```
<?php
header("Content-Type: application/pdf");
header("Content-Disposition: attachment; filename='sample.pdf'");
readfile("sample.pdf");
?>
```

☞ Forces the browser to download `sample.pdf` instead of opening it.

d) Controlling Caching

```
<?php
header("Cache-Control: no-cache, must-revalidate");
header("Expires: Sat, 1 Jan 2000 00:00:00 GMT");
?>
```

☞ Prevents browser from caching the page.

e) Sending HTTP Status Codes

```
<?php
```

```
header("HTTP/1.1 404 Not Found");
echo "Page not found!";
?>
```

Q9. Explain the difference between include and require in PHP

1. Purpose of `include` and `require`

- Both are used to **reuse code** across multiple PHP files.
- Commonly used for headers, footers, database connections, or configuration files.

Example:

```
// main.php
include "header.php";
require "config.php";

echo "Main content here.";
include "footer.php";
```

2. Key Differences

Feature	include	require
Error Handling	Produces a warning if the file is missing, but the script continues execution.	Produces a fatal error if the file is missing, and the script stops execution .
Use Case	Use when the file is optional (e.g., sidebar, banner).	Use when the file is mandatory (e.g., database config, essential classes).
Performance	Slightly faster in non-critical files (since script continues).	Ensures critical files are present, otherwise stops script.

3. Examples

Example with `include`

```
<?php
echo "Before include<br>";
include "non_existing_file.php"; // File not found → Warning only
echo "After include<br>";
?>
```

🔗 Output:

```
Before include
Warning: include(non_existing_file.php): Failed to open stream...
After include
```

(Script continues execution ✓)

Example with `require`

```
<?php
echo "Before require<br>";
require "non_existing_file.php"; // File not found → Fatal error
echo "After require<br>";
?>
```

🔗 Output:

```
Before require
Fatal error: require(): Failed opening required 'non_existing_file.php'
```

(Script stops execution ✗)

4. Variants

- `include_once` → Includes file only once, ignores duplicate includes.
- `require_once` → Requires file only once, avoids fatal error on duplicate load.

Q10. Explain what PHP expressions are and give examples of arithmetic and logical operations.

- An **expression** in PHP is **anything that produces a value**.
- It can be a simple value (like 5, "Hello"), a variable (\$x), or a combination of variables, operators, and functions.
- The result of an expression can be stored in a variable or used directly.

✓ Example:

```
<?php
```

```
$x = 10 + 5;    // expression (10 + 5) produces 15
$y = $x * 2;    // expression ($x * 2) produces 30
echo $y;        // 30
?>
```

2. Arithmetic Expressions

These use **arithmetic operators** to perform mathematical calculations.

Operators:

- + → Addition
- - → Subtraction
- * → Multiplication
- / → Division
- % → Modulus (remainder)
- ** → Exponentiation (power)

✓ Example:

```
<?php
$a = 15;
$b = 4;

echo $a + $b; // 19
echo $a - $b; // 11
echo $a * $b; // 60
echo $a / $b; // 3.75
echo $a % $b; // 3 (remainder)
echo $a ** $b; // 50625 (15^4)
?>
```

3. Logical Expressions

These use **logical operators** to combine conditions (true/false values).

Operators:

- && (AND) → True if both conditions are true.
- || (OR) → True if at least one condition is true.
- ! (NOT) → Negates a condition.
- xor → True if only one condition is true.

✓ Example:

```
<?php
$x = 10;
$y = 20;

var_dump($x > 5 && $y > 15); // true (both true)
var_dump($x > 15 || $y > 15); // true (second true)
var_dump(!$x > 15); // true (because $x > 15 is false)
var_dump($x > 5 xor $y < 15); // true (only first condition true)
?>
```